

SUPPORT DE COURS COMPLET

IA appliquée à la sécurité informatique

16h CM · 8h TP · 24h

Dr. Zakaria Sawadogo

École Polytechnique de Ouagadougou



Sommaire

Syllabus

1. Chapitre 1 — Introduction : IA et cybersécurité, panorama
2. Chapitre 2 — Rappels de machine learning
3. Chapitre 3 — Détection d'anomalies et d'intrusions par ML
4. Chapitre 4 — Détection de malwares par machine learning
5. Chapitre 5 — NLP appliqué à la sécurité (phishing, analyse de logs)
6. Chapitre 6 — Sécurité de l'IA : attaques adversariales et IA offensive

IA appliquée à la sécurité informatique

Volume horaire : 16h CM · 8h TP (24h)

Objectifs pédagogiques

- Comprendre les apports et les limites de l'intelligence artificielle et du machine learning pour la cybersécurité défensive (détection d'intrusions, de malwares, de phishing).
- Mettre en œuvre des modèles de machine learning simples sur des données de sécurité (réseau, logs, texte).
- Comprendre les risques spécifiques à l'IA elle-même en tant que surface d'attaque (attaques adversariales, empoisonnement de données) et les usages offensifs de l'IA.

Prérequis

Bases de programmation (le module *Algorithmique et Programmation en C* apporte les fondamentaux de logique algorithmique ; les TP de ce module utilisent Python) et notions de statistiques élémentaires.

Plan de séances

Cours magistraux (16h)

Séance	Chapitre	Durée
1	Introduction : IA et cybersécurité, panorama	2h
2	Rappels de machine learning	3h
3	Détection d'anomalies et d'intrusions par ML	3h
4	Détection de malwares par machine learning	2h
5	NLP appliqué à la sécurité (phishing, logs)	3h
6	Sécurité de l'IA : attaques adversariales et IA offensive	3h

Travaux pratiques (8h)

Séance	Sujet	Durée
1	Classification de trafic réseau (NSL-KDD)	2h
2	Détection d'anomalies non supervisée	2h
3	Détection de phishing par NLP	2h

Séance	Sujet	Durée
4	Attaque adversariale sur un classifieur	2h

Évaluation

- TP notés : 40 %
- Projet (application d'un modèle ML à un jeu de données de sécurité au choix) : 30 %
- Examen final : 30 %

Environnement technique

Python 3, `scikit-learn`, `pandas`, `numpy`, `matplotlib` ; `PyTorch` ou `TensorFlow` pour les exercices avancés (TP4). Jeux de données publics utilisés : NSL-KDD (intrusions réseau), un corpus public d'e-mails de phishing.

Bibliographie

- S. Raschka, V. Mirjalili, *Python Machine Learning*.
- I. Goodfellow, Y. Bengio, A. Courville, *Deep Learning*.
- OWASP Machine Learning Security Top 10.
- MITRE ATLAS (Adversarial Threat Landscape for Artificial-Intelligence Systems).

Chapitre 1 — Introduction : IA et cybersécurité, panorama

1. Pourquoi l'IA en cybersécurité ?

Les volumes de données à surveiller (journaux, flux réseau, alertes) ont dépassé depuis longtemps la capacité d'analyse manuelle humaine. L'IA et le machine learning (ML) permettent d'automatiser la détection de schémas (patterns) suspects, de traiter des volumes massifs en temps quasi réel, et de détecter des menaces nouvelles (« zero-day ») que des règles de détection statiques (signatures) ne couvrent pas encore.

Trois facteurs expliquent cette montée en puissance de l'IA dans les outils de sécurité :

- **L'explosion du volume de données** : un centre opérationnel de sécurité (SOC) de taille moyenne peut recevoir des milliers, voire des millions d'événements par jour (connexions réseau, authentications, alertes d'antivirus). Aucune équipe humaine ne peut examiner ce volume ligne par ligne.
- **La disponibilité croissante de puissance de calcul et de données** : l'entraînement de modèles complexes (réseaux de neurones profonds notamment) est devenu accessible grâce aux progrès matériels (GPU) et à la disponibilité de jeux de données publics ou semi-publics (trafic réseau capturé, corpus de malwares, corpus de phishing).
- **L'évolution des attaquants eux-mêmes** : les techniques d'attaque se diversifient et s'automatisent, ce qui pousse la défense à automatiser également ses capacités de détection et de réponse.

Il est toutefois essentiel de garder à l'esprit que l'IA en sécurité est un **outil d'aide à la décision et d'automatisation**, pas une boîte magique. Elle vient compléter des dispositifs existants (pare-feux, IDS/IPS par signatures, SIEM, procédures organisationnelles), et sa valeur dépend directement de la qualité de son intégration dans un processus de sécurité plus large.

1.1 Bref rappel : IA, machine learning, deep learning

Ces trois termes sont souvent utilisés de façon interchangeable dans les discours commerciaux, ce qui entretient une confusion qu'il convient de dissiper dès l'introduction du module :

- **Intelligence artificielle (IA)** : discipline englobante visant à faire réaliser par une machine des tâches nécessitant normalement une forme d'intelligence (raisonnement, perception, apprentissage). Elle inclut aussi des approches non statistiques (systèmes experts à base de règles, par exemple).
- **Machine learning (ML)** : sous-ensemble de l'IA où le système apprend automatiquement des régularités à partir de données, plutôt que d'exécuter des règles écrites explicitement par un humain. C'est le cœur de ce module.
- **Deep learning (apprentissage profond)** : sous-ensemble du ML reposant sur des réseaux de neurones à plusieurs couches (« profonds »), particulièrement performants sur des données non

structurées (image, texte, séquences) mais plus coûteux en données et en calcul, et généralement moins interprétables.

En cybersécurité, on trouve les trois : des règles expertes (signatures, règles YARA, règles de corrélation SIEM), du ML « classique » (Random Forest, régression logistique) et du deep learning (réseaux de neurones pour l'analyse de séquences d'appels système ou de texte). Le chapitre 2 détaille les différences entre ces familles d'algorithmes.

2. Deux faces d'une même discipline

- **IA défensive** : détection d'intrusions, de malwares, de phishing, priorisation d'alertes, automatisation de la réponse à incident (SOAR).
- **IA comme surface d'attaque** : les systèmes d'IA eux-mêmes deviennent des cibles (empoisonnement de données d'entraînement, attaques adversariales, extraction de modèle).
- **IA offensive** : usage de l'IA par les attaquants (génération de contenus de phishing convaincants, automatisation de la reconnaissance, génération de code malveillant).

Ce module couvre les trois dimensions, avec un accent particulier sur la défense (chapitres 3 à 5) puis sur la sécurité de l'IA elle-même (chapitre 6).

Il est utile de visualiser ces trois dimensions comme les sommets d'un même triangle, en interaction permanente : une amélioration de la détection ML (dimension défensive) pousse les attaquants à développer des techniques d'évasion visant spécifiquement les modèles (dimension « IA comme surface d'attaque »), tandis que l'IA offensive leur permet d'automatiser et de personnaliser ces contournements à grande échelle. Comprendre une seule de ces dimensions sans les deux autres donne une vision incomplète, voire dangereuse, du sujet — un analyste qui déploie un détecteur ML sans connaître les techniques d'évasion (chapitre 6) surestimera la robustesse de son dispositif.

3. Cas d'usage défensifs courants

Cas d'usage	Type de tâche ML	Exemple
Détection d'intrusions réseau	classification supervisée / détection d'anomalies	identifier un trafic réseau anormal
Détection de malwares	classification supervisée	classer un fichier binaire comme sain/malveillant
Détection de phishing	classification de texte (NLP)	analyser le contenu d'un e-mail ou d'une URL
Analyse comportementale des utilisateurs (UEBA)	détection d'anomalies	repérer un compte utilisateur au comportement inhabituel
Priorisation d'alertes SOC	classification / scoring	réduire la fatigue d'alerte des analystes

3.1 Où l'IA s'intègre-t-elle concrètement dans une architecture de sécurité ?

Il est important de ne pas se représenter l'IA comme un produit unique, mais comme une brique intégrée à des outils existants :

- **Au niveau du réseau** : les IDS/IPS de nouvelle génération intègrent des modules de détection d'anomalies en complément des signatures classiques (chapitre 3).
- **Au niveau du poste de travail** : les EDR (*Endpoint Detection and Response*) embarquent des classifieurs ML entraînés sur des caractéristiques statiques et comportementales des exécutable (chapitre 4).
- **Au niveau de la messagerie** : les passerelles anti-phishing combinent des modèles NLP et des règles techniques sur les URL et en-têtes (chapitre 5).
- **Au niveau du SOC** : les plateformes SIEM/SOAR utilisent le ML pour corrélérer les alertes, réduire le bruit, et parfois pour suggérer des playbooks de réponse automatisée.

Ainsi, un même diplômé du programme de cybersécurité amené à travailler comme analyste SOC, pentester, ou architecte sécurité rencontrera très probablement des composants ML dans les outils qu'il opère, même sans les avoir lui-même développés — d'où l'importance de comprendre leur fonctionnement et leurs limites, indépendamment de la spécialisation professionnelle visée.

4. Limites et illusions à éviter

- **Le ML n'élimine pas le besoin d'expertise humaine** : un modèle mal conçu ou mal évalué produit des faux positifs (fatigue d'alerte) ou des faux négatifs (menaces non détectées) coûteux.
- **Les données d'entraînement conditionnent tout** : un modèle entraîné sur un contexte donné (réseau, période) généralise mal à un contexte différent (concept drift).
- **Explicabilité** : un modèle « boîte noire » (ex. réseau de neurones profond) est difficile à auditer et à justifier auprès d'un analyste ou d'un régulateur — un critère important dans le choix du modèle selon le contexte d'usage.
- **L'IA n'est pas une solution miracle contre un attaquant déterminé** : un attaquant conscient qu'un système de détection utilise du ML peut chercher spécifiquement à le contourner (chapitre 6).

4.1 Le mythe du modèle « universel »

Une idée reçue fréquente consiste à croire qu'un modèle ML performant, une fois entraîné, fonctionnera durablement et dans tout contexte. En pratique, un modèle de détection déployé dans un environnement de production est soumis à plusieurs formes d'usure :

- **La dérive conceptuelle (concept drift)** évoquée plus haut : les usages légitimes évoluent (nouvelles applications, nouveaux protocoles), ce qui rapproche progressivement le trafic « normal » de ce que le modèle avait appris à considérer comme suspect, dégradant le rappel ou augmentant le taux de faux positifs.
- **L'adaptation active de l'attaquant** : contrairement à un phénomène purement statistique (comme la météo), l'« environnement » en sécurité comprend des acteurs qui réagissent délibérément aux défenses mises en place — un comportement qu'on ne retrouve pas dans la

plupart des applications « classiques » du ML (recommandation, vision par ordinateur sur des images naturelles, etc.).

- **Le changement d'échelle** : un modèle validé sur un petit jeu de données pilote peut se comporter très différemment une fois déployé à l'échelle d'un réseau d'entreprise complet, avec une diversité de trafic bien plus grande.

Ce constat justifie une pratique centrale du domaine : la **supervision continue** des modèles en production (réévaluation périodique, réentraînement, suivi de métriques de dérive), abordée plus en détail au chapitre 2.

4.2 Coût, gouvernance et responsabilité

Au-delà des aspects strictement techniques, l'adoption de l'IA en sécurité soulève des questions de gouvernance qu'un futur professionnel doit savoir anticiper :

- **Qui est responsable en cas d'erreur du modèle ?** (un faux négatif ayant laissé passer une attaque, un faux positif ayant bloqué une activité légitime critique).
- **Comment documenter et auditer** les décisions d'un modèle, en particulier dans des secteurs réglementés (finance, santé) ?
- **Quel est le coût réel** (données, calcul, expertise, maintenance) d'un système ML par rapport à une approche par règles plus simple, et ce coût est-il justifié par le gain de détection obtenu ?

Ces questions n'ont pas de réponse unique : elles dépendent du contexte organisationnel et seront reprises, sous l'angle technique de la sécurité de l'IA elle-même, au chapitre 6.

5. Articulation avec les autres modules du programme

Ce module s'appuie sur les notions de trafic réseau et de vulnérabilités vues en *Audit organisation et technique*, et prépare une lecture critique des outils de sécurité modernes (EDR, XDR, SIEM) qui intègrent aujourd'hui massivement des composants de ML.

Concrètement, les compétences développées dans les modules précédents restent indispensables : comprendre un protocole réseau (pour interpréter les caractéristiques extraites d'un flux), connaître les techniques d'attaque classiques (pour évaluer si un modèle de détection couvre réellement les menaces pertinentes), et maîtriser les principes d'audit (pour évaluer la fiabilité et les limites d'un système de détection avant de s'y fier en production). L'IA appliquée à la sécurité n'est donc pas un domaine cloisonné, mais un prolongement naturel des compétences déjà acquises, appliqué à un nouvel outil.

À retenir

- L'IA en cybersécurité recouvre trois dimensions : défense, sécurité de l'IA elle-même, et usage offensif par les attaquants.
- Le ML complète mais ne remplace pas l'expertise humaine et les approches par signatures/règles.

- La qualité et la représentativité des données d'entraînement sont le facteur déterminant de la performance réelle d'un système de détection basé ML.
- Un modèle performant en laboratoire peut se dégrader en production à cause de la dérive conceptuelle et de l'adaptation active des attaquants ; la supervision continue est indispensable.
- L'adoption de l'IA en sécurité pose aussi des questions de gouvernance (responsabilité, auditabilité, coût) qui dépassent le seul aspect technique et seront reprises tout au long du module.

Chapitre 2 — Rappels de machine learning

1. Apprentissage supervisé, non supervisé, par renforcement

Type	Principe	Exemple en sécurité
Supervisé	apprend à partir d'exemples étiquetés (entrée, sortie attendue)	classifier un fichier comme malware/sain à partir d'exemples déjà étiquetés
Non supervisé	découvre une structure dans des données non étiquetées	regrouper des connexions réseau similaires, détecter des anomalies
Par renforcement	un agent apprend par essai-erreur en maximisant une récompense	automatisation de la réponse à incident (usage encore émergent)

1.1 Pourquoi cette distinction est centrale en sécurité

Le choix entre ces trois paradigmes n'est pas qu'une question académique : il est directement contraint par la disponibilité de données étiquetées, qui pose un problème récurrent en sécurité.

- Pour entraîner un modèle **supervisé** de détection de malwares, il faut disposer d'un grand nombre d'exemples déjà étiquetés « malveillant » ou « sain ». C'est possible grâce à des bases partagées par la communauté (échantillons soumis à des services d'analyse), mais ces exemples sont nécessairement **rétrospectifs** : ils décrivent des menaces déjà connues, jamais les attaques de demain.
- Les approches **non supervisées** (clustering, détection d'anomalies) ne nécessitent pas d'étiquettes : elles apprennent uniquement à partir d'une notion de « normalité » observée dans les données. C'est pourquoi elles sont privilégiées pour détecter des menaces inédites (chapitre 3), au prix d'un taux de fausses alertes généralement plus élevé.
- L'**apprentissage semi-supervisé**, qui combine un petit nombre d'exemples étiquetés avec un grand volume de données non étiquetées, est une piste intermédiaire de plus en plus explorée en sécurité, où l'étiquetage manuel par des experts est coûteux et lent.
- L'apprentissage **par renforcement** reste marginal en production dans la détection elle-même, mais gagne du terrain pour l'automatisation de la réponse à incident (choisir automatiquement une action de remédiation en fonction du contexte) et pour la génération de scénarios d'attaque simulés (« red teaming » automatisé).

2. Classification vs régression

- **Classification** : prédire une catégorie (ex. « malveillant » / « bénin »).
- **Régression** : prédire une valeur continue (ex. score de risque entre 0 et 100).

La plupart des cas d'usage en détection de sécurité relèvent de la classification (binaire ou multi-classe).

Il existe cependant des variantes utiles à connaître :

- La **classification multi-classe** ne se contente pas de dire « malveillant/sain » : elle peut viser à identifier la *famille* de malware (ransomware, cheval de Troie, ver, etc.), ce qui aide à prioriser la réponse.
- La **classification multi-label** autorise plusieurs étiquettes simultanées pour une même observation (un même binaire peut à la fois relever d'un cheval de Troie *et* d'un comportement de vol d'identifiants).
- Certains systèmes combinent classification et régression : un modèle de scoring de risque (régression, valeur continue) est ensuite converti en décision binaire (classification) via un **seuil de décision** ajustable — un point essentiel car ce seuil détermine directement le compromis entre précision et rappel (section 5).

3. Cycle de vie d'un projet de machine learning

1. **Collecte et préparation des données** : nettoyage, gestion des valeurs manquantes, normalisation.
2. **Ingénierie des caractéristiques (feature engineering)** : transformer des données brutes (paquets réseau, texte) en variables numériques exploitables par un modèle.
3. **Séparation des données** : ensemble d'entraînement, de validation, de test — pour éviter le surapprentissage (*overfitting*).
4. **Entraînement du modèle**.
5. **Évaluation** sur des données jamais vues par le modèle.
6. **Déploiement et supervision continue** (un modèle en production doit être réévalué régulièrement : concept drift).

3.1 Illustration avec un pipeline scikit-learn simplifié

L'extrait ci-dessous illustre, de façon pédagogique, l'enchaînement des étapes 2 à 5 sur un jeu de données fictif de connexions réseau caractérisées par quelques variables numériques (durée, nombre d'octets échangés) et une variable catégorielle (protocole). Il ne s'agit pas d'un exemple destiné à être exécuté sur des données réelles, mais d'illustrer la structure typique d'un pipeline :

```

import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler, OneHotEncoder
from sklearn.compose import ColumnTransformer
from sklearn.pipeline import Pipeline
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import classification_report

# Jeu de données fictif : chaque ligne = une connexion réseau
# 'label' = 1 si connexion malveillante, 0 sinon (exemple pédagogique)
df = pd.read_csv("connexions_reseau_exemple.csv")

X = df.drop(columns=["label"])
y = df["label"]

# Étape 3 : séparation entraînement / test, en conservant
# la proportion de classes (important sur données déséquilibrées)
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y
)

# Étape 2 : prétraitement des caractéristiques
# - variables numériques : normalisation
# - variable catégorielle (protocole) : encodage one-hot
preprocesseur = ColumnTransformer([
    ("num", StandardScaler(), ["duree", "octets_src", "octets_dst"]),
    ("cat", OneHotEncoder(handle_unknown="ignore"), ["protocole"]),
])

pipeline = Pipeline([
    ("prep", preprocesseur),
    ("modele", RandomForestClassifier(
        n_estimators=200, class_weight="balanced", random_state=42
    )),
])

# Étape 4 : entraînement
pipeline.fit(X_train, y_train)

# Étape 5 : évaluation sur des données jamais vues à l'entraînement
y_pred = pipeline.predict(X_test)
print(classification_report(y_test, y_pred, target_names=["bénin", "malveillant"]))

```

Quelques remarques pédagogiques sur cet extrait :

- L'option `stratify=y` de `train_test_split` garantit que la proportion d'exemples malveillants est similaire dans les ensembles d'entraînement et de test — un point important car, comme discuté en section 5, les classes sont souvent très déséquilibrées en sécurité.
- L'usage d'un `Pipeline` scikit-learn regroupe prétraitement et modèle en un seul objet : cela évite une erreur fréquente en pratique, la **fuite de données (data leakage)**, qui consiste par exemple à calculer la normalisation (moyenne, écart-type) sur l'ensemble complet des données *avant* la séparation train/test, ce qui laisse involontairement filtrer de l'information de l'ensemble de test vers l'entraînement et surestime la performance réelle du modèle.

- Le paramètre `class_weight="balanced"` demande au modèle de pénaliser davantage les erreurs sur la classe minoritaire (malveillant), une réponse simple — mais pas toujours suffisante — au déséquilibre des classes.

3.2 Validation croisée : au-delà d'une seule séparation train/test

Une unique séparation entraînement/test peut donner une estimation optimiste ou pessimiste de la performance selon le hasard du découpage, en particulier sur un jeu de données de taille modeste. La **validation croisée à k plis (k-fold cross-validation)** répète l'entraînement et l'évaluation k fois sur des découpages différents, puis moyenne les résultats, donnant une estimation plus robuste :

```
from sklearn.model_selection import cross_val_score

scores = cross_val_score(pipeline, X, y, cv=5, scoring="f1")
print(f"F1-score moyen sur 5 plis : {scores.mean():.3f} (écart-type {scores.std():.3f})")
```

En sécurité, une variante appelée **validation croisée temporelle** est souvent préférable à un découpage aléatoire classique : les plis d'entraînement doivent précéder chronologiquement les plis d'évaluation (par exemple via `TimeSeriesSplit` de scikit-learn), afin de simuler fidèlement une situation réelle de déploiement, où le modèle est entraîné sur du trafic passé et évalué sur du trafic futur — un découpage purement aléatoire pourrait sinon entraîner le modèle sur des données « du futur » par rapport à son test, ce qui gonflerait artificiellement la performance estimée.

4. Quelques algorithmes classiques

Algorithme	Type	Caractéristique
Régression logistique	supervisé, classification	simple, interprétable, bonne base de référence
Arbre de décision / Forêt aléatoire (Random Forest)	supervisé, classification/régression	robuste, gère bien les données hétérogènes, relativement interprétable
SVM (Support Vector Machine)	supervisé, classification	efficace en haute dimension
K-means	non supervisé, clustering	regroupe des observations similaires
Isolation Forest	non supervisé, détection d'anomalies	isole les observations atypiques efficacement, utilisé en TP2
Réseaux de neurones / Deep Learning	supervisé ou non supervisé	performant sur données complexes (texte, image, séquences) mais moins interprétable

4.1 Intuition sur quelques algorithmes clés

Comprendre le principe géométrique ou statistique sous-jacent aide à choisir le bon algorithme pour un problème donné, plutôt que de traiter chaque modèle comme une boîte noire interchangeable :

- **Régression logistique** : modélise la probabilité qu'une observation appartienne à la classe positive comme une fonction sigmoïde d'une combinaison linéaire des variables d'entrée. Son atout majeur est l'interprétabilité : le poids associé à chaque variable indique directement, toutes choses égales par ailleurs, son influence sur la prédiction — un avantage précieux pour justifier une alerte auprès d'un analyste SOC.
- **Arbres de décision et forêts aléatoires** : un arbre de décision découpe successivement l'espace des caractéristiques par des règles simples (« si `octets_dst > 5000` alors... »), ce qui le rend très lisible mais sujet au surapprentissage sur un arbre unique. Une **forêt aléatoire** entraîne un grand nombre d'arbres sur des sous-échantillons différents des données et des variables, puis agrège leurs prédictions (vote majoritaire) : elle gagne en robustesse et en performance ce qu'elle perd en lisibilité immédiate, tout en restant plus interprétable qu'un réseau de neurones (on peut par exemple mesurer l'**importance des variables**).
- **SVM** : cherche l'hyperplan séparant les deux classes avec la marge maximale, éventuellement après projection dans un espace de dimension supérieure via une fonction noyau (*kernel*) pour traiter des frontières non linéaires. Efficace lorsque le nombre de variables est grand par rapport au nombre d'observations, mais son entraînement passe mal à l'échelle sur de très gros volumes de données, ce qui limite son usage sur des flux réseau massifs.
- **K-means** : partitionne les observations en k groupes en minimisant la distance de chaque point au centre (centroïde) de son groupe. Simple et rapide, mais suppose des groupes de forme sphérique et de taille comparable, et nécessite de choisir k à l'avance — une limite importante en détection d'anomalies, où le nombre de comportements distincts n'est pas connu d'avance.
- **Isolation Forest** : contrairement aux approches basées sur une notion de distance ou de densité, elle exploite l'idée qu'une observation atypique est plus facile à « isoler » du reste des données par des séparations aléatoires successives (elle nécessite en moyenne moins de coupures pour être isolée seule dans une branche de l'arbre). Cette propriété en fait un algorithme rapide et bien adapté à la détection d'anomalies en haute dimension, ce qui explique son usage fréquent en sécurité (approfondi au chapitre 3 et pratiqué en TP2).
- **Réseaux de neurones** : composés de couches de neurones artificiels reliés par des poids ajustés lors de l'entraînement (rétropropagation du gradient), ils peuvent approximer des fonctions très complexes et non linéaires. Leur force réside dans le traitement de données non structurées (texte brut, séquences d'appels système, images), mais ils nécessitent généralement plus de données et de puissance de calcul, et leur fonctionnement interne reste largement opaque — un inconvénient dans un contexte où l'explicabilité d'une décision de sécurité peut être requise.

4.2 Comment choisir un algorithme en pratique ?

Il n'existe pas d'algorithme universellement supérieur (« no free lunch theorem ») : le choix dépend du contexte. Quelques critères pratiques à considérer, particulièrement pertinents en sécurité :

- **Volume et nature des données disponibles** : peu de données étiquetées → privilégier des modèles simples (régression logistique, arbres) moins sujets au surapprentissage ; beaucoup de données non structurées (texte, séquences) → le deep learning devient compétitif.
- **Besoin d'explicabilité** : un contexte réglementaire ou une nécessité de justifier une alerte auprès d'un analyste pousse vers des modèles interprétables (arbres, régression logistique) ou

vers l'usage de techniques d'explicabilité post-hoc (par exemple SHAP) sur des modèles plus complexes.

- **Contraintes de latence** : un IDS traitant du trafic en temps réel ne peut pas se permettre un modèle dont l'inférence est trop lente ; un modèle plus simple, quitte à être légèrement moins performant, peut être préférable en production.
- **Robustesse aux données déséquilibrées** : certains algorithmes (forêts aléatoires, gradient boosting) offrent des mécanismes natifs de pondération des classes plus simples à exploiter que d'autres.

5. Métriques d'évaluation, essentielles en sécurité

Pour une classification binaire, la matrice de confusion distingue :

	Prédit positif	Prédit négatif
Réel positif	Vrai Positif (VP)	Faux Négatif (FN)
Réel négatif	Faux Positif (FP)	Vrai Négatif (VN)

- **Précision** = $VP / (VP + FP)$: parmi les alertes levées, quelle proportion est réellement une menace ?
- **Rappel (recall)** = $VP / (VP + FN)$: parmi les menaces réelles, quelle proportion a été détectée ?
- **F1-score** : moyenne harmonique de précision et rappel.
- **Taux de faux positifs** = $FP / (FP + VN)$: critique en sécurité, car un taux élevé provoque la « fatigue d'alerte » des analystes, qui finissent par ignorer les alertes.

En sécurité, la simple exactitude globale (accuracy) est souvent trompeuse : sur un jeu de données très déséquilibré (ex. 0,1 % de trafic réellement malveillant), un modèle qui prédit toujours « bénin » atteint 99,9 % d'exactitude tout en étant totalement inutile. D'où l'importance du rappel et de la précision, en particulier sur la classe minoritaire (l'attaque).

5.1 Le compromis précision/rappel et le rôle du seuil de décision

La plupart des classifieurs ne produisent pas directement une décision binaire, mais une probabilité (ou un score) qu'une observation appartienne à la classe positive. Cette probabilité est ensuite comparée à un **seuil de décision** (souvent 0,5 par défaut) pour produire l'étiquette finale. Faire varier ce seuil déplace le compromis entre précision et rappel :

- **Abaisser le seuil** (déclencher une alerte plus facilement) augmente généralement le rappel (moins de menaces manquées) mais diminue la précision (davantage de fausses alertes).
- **Relever le seuil** fait l'inverse : moins de fausses alertes, mais un risque accru de laisser passer de véritables menaces.

Le choix du seuil optimal dépend du **coût relatif** des deux types d'erreur dans le contexte opérationnel considéré : dans un SOC déjà saturé d'alertes, un seuil plus élevé peut être préféré pour limiter la fatigue d'alerte, quitte à accepter un rappel légèrement inférieur ; à l'inverse, sur un système critique

(infrastructure de santé, contrôle industriel), un rappel maximal peut être priorisé malgré un volume d'alertes plus important à traiter.

Deux outils graphiques aident à visualiser ce compromis sur l'ensemble des seuils possibles, plutôt qu'à un seuil fixé arbitrairement :

- La **courbe ROC** (*Receiver Operating Characteristic*) trace le taux de vrais positifs en fonction du taux de faux positifs pour tous les seuils, et l'**AUC** (aire sous la courbe) résume la capacité globale du modèle à séparer les classes, indépendamment d'un seuil particulier.
- La **courbe précision-rappel** est généralement plus informative que la courbe ROC lorsque les classes sont très déséquilibrées, situation fréquente en sécurité, car elle reste sensible aux performances sur la classe rare (l'attaque).

```
from sklearn.metrics import precision_recall_curve, roc_auc_score
import numpy as np

# proba : probabilité prédite d'appartenance à la classe "malveillant"
proba = pipeline.predict_proba(X_test)[: , 1]

precisions, rappels, seuils = precision_recall_curve(y_test, proba)
print("AUC ROC :", roc_auc_score(y_test, proba))

# Exemple : choisir le seuil qui maximise le F1-score sur l'ensemble de validation
f1_scores = 2 * (precisions * rappels) / (precisions + rappels + 1e-9)
meilleur_seuil = seuils[np.argmax(f1_scores)]
print("Seuil retenu :", meilleur_seuil)
```

5.2 Autres métriques utiles en contexte opérationnel

Au-delà de la précision, du rappel et du F1-score, un praticien de la sécurité gagne à connaître :

- **Le taux de fausses alertes en volume absolu** (et non seulement en proportion) : sur un très grand volume d'événements, même un taux de faux positifs de 0,1 % peut représenter des centaines d'alertes quotidiennes à traiter manuellement — une information que le pourcentage seul ne rend pas visible.
- **Le délai de détection (time to detect)** : au-delà de la seule justesse de la prédiction, le temps écoulé entre le début d'une attaque et sa détection est un critère opérationnel de premier plan, en particulier pour les attaques à progression rapide (ransomware).
- **La matrice de confusion multi-classe**, lorsque le problème dépasse la simple distinction bénin/malveillant (ex. identification de la famille de malware), permettant d'identifier quelles classes sont le plus souvent confondues entre elles.

6. Surapprentissage et sous-apprentissage

- **Surapprentissage (overfitting)** : le modèle « apprend par cœur » les données d'entraînement et généralise mal à de nouvelles données.
- **Sous-apprentissage (underfitting)** : le modèle est trop simple pour capturer la structure des données.

La validation croisée (cross-validation) et un ensemble de test totalement indépendant permettent de détecter ces situations avant tout déploiement.

6.1 Diagnostiquer et corriger ces situations

En pratique, on distingue le surapprentissage et le sous-apprentissage en comparant la performance du modèle sur l'ensemble d'entraînement et sur l'ensemble de validation :

- Une performance **élevée à l'entraînement mais nettement plus faible en validation** est le symptôme classique du surapprentissage : le modèle a mémorisé des particularités propres à l'échantillon d'entraînement (y compris du bruit) qui ne se généralisent pas.
- Une performance **médiocre à la fois à l'entraînement et en validation** signale un sous-apprentissage : le modèle est trop contraint (trop simple, ou insuffisamment entraîné) pour capturer la structure réelle des données.

Quelques leviers pratiques pour remédier à ces situations, fréquemment combinés :

- Contre le **surapprentissage** : simplifier le modèle (réduire la profondeur d'un arbre, le nombre de couches d'un réseau de neurones), augmenter le volume de données d'entraînement, utiliser une **régularisation** (pénaliser les modèles trop complexes, par exemple via les pénalités L1/L2 en régression logistique), ou appliquer un **arrêt précoce** (*early stopping*) pendant l'entraînement d'un réseau de neurones.
- Contre le **sous-apprentissage** : complexifier le modèle, enrichir l'ingénierie des caractéristiques (chapitres 3 à 5), ou entraîner plus longtemps.

Une bonne pratique consiste à tracer les **courbes d'apprentissage** (performance en fonction de la taille de l'ensemble d'entraînement, ou du nombre d'itérations) pour visualiser directement lequel de ces deux régimes s'applique, plutôt que de deviner uniquement à partir d'un score final.

À retenir

- Le choix entre supervisé/non supervisé dépend de la disponibilité de données étiquetées, souvent rare en sécurité pour les attaques nouvelles.
- Précision, rappel et F1-score, pas la seule exactitude, sont les métriques pertinentes pour évaluer un modèle de détection en contexte déséquilibré.
- Le seuil de décision permet d'ajuster le compromis précision/rappel selon le coût opérationnel des faux positifs et des faux négatifs dans le contexte visé.
- Le cycle de vie complet (données → features → entraînement → évaluation → supervision continue) doit être maîtrisé, pas seulement l'entraînement du modèle.
- En sécurité, la validation croisée temporelle est souvent préférable à un découpage aléatoire, afin de simuler fidèlement un déploiement réel sur des données futures inconnues du modèle.
- Le surapprentissage et le sous-apprentissage se diagnostiquent en comparant performance à l'entraînement et en validation, et se corrigent par des leviers différents et parfois opposés.

Chapitre 3 — Détection d'anomalies et d'intrusions par ML

1. Détection par signatures vs détection par anomalies

Approche	Principe	Avantage	Limite
Signatures	comparaison à des motifs d'attaques connues	très faible taux de faux positifs sur les menaces connues	ne détecte pas les attaques inédites
Anomalies (ML)	apprentissage du comportement « normal », alerte sur tout écart significatif	peut détecter des menaces nouvelles (zero-day)	taux de faux positifs souvent plus élevé, nécessite une phase d'apprentissage du « normal »

En pratique, les IDS/IPS (systèmes de détection/prévention d'intrusion) modernes combinent les deux approches.

1.1 Un troisième pilier : la détection par spécification

Une troisième approche, moins souvent mise en avant mais utile à connaître, complète ce panorama : la **détection par spécification (specification-based)**. Elle consiste à définir manuellement les comportements *autorisés* d'un système (par exemple, les commandes qu'un protocole industriel est censé accepter) et à considérer comme suspect tout ce qui s'en écarte. Contrairement à la détection par anomalies, elle ne repose pas sur un apprentissage statistique mais sur une modélisation experte du comportement légitime — elle est donc moins sujette aux faux positifs liés au bruit statistique, mais coûteuse à maintenir et peu adaptable à des systèmes complexes ou évolutifs. Elle reste néanmoins pertinente dans des environnements très contraints (systèmes industriels SCADA, objets connectés à fonction unique) où le comportement légitime est simple à spécifier exhaustivement.

1.2 Pourquoi la détection par anomalies séduit malgré son coût en faux positifs

Le principal argument en faveur de la détection par anomalies, malgré ses limites, tient à la nature même des attaques modernes : une part croissante des compromissions n'exploite pas de vulnérabilité logicielle « signable » au sens classique, mais des techniques dites *living-off-the-land* (utilisation d'outils légitimes déjà présents sur le système à des fins malveillantes) ou des identifiants compromis via ingénierie sociale. Dans ces cas, il n'existe littéralement aucune signature statique à détecter : seul un écart de comportement (un compte utilisateur se connectant à une heure inhabituelle, un outil d'administration exécuté depuis un poste qui ne l'utilise jamais) peut révéler l'intrusion. C'est ce constat qui justifie l'investissement dans des approches par anomalies malgré le coût opérationnel plus élevé en gestion des fausses alertes.

2. Sources de données pour un IDS basé ML

- **Trafic réseau** : caractéristiques dérivées des flux (NetFlow) : durée de connexion, protocole, nombre d'octets échangés, nombre de paquets, taux d'erreur.
- **Journaux systèmes/applicatifs** : séquences d'événements, fréquence des connexions échouées.
- **Comportement utilisateur (UEBA)** : horaires de connexion habituels, ressources accédées, volume de données téléchargées.

2.1 Granularité de l'analyse : paquet, flux ou session

Le choix de la granularité d'analyse conditionne fortement le type de menaces détectables et le coût de calcul associé :

- **Analyse par paquet (deep packet inspection)** : la plus fine, elle permet d'inspecter le contenu même des paquets, mais génère un volume de données considérable et pose des questions de confidentialité (inspection du contenu, parfois chiffré).
- **Analyse par flux (NetFlow/IPFIX)** : agrège les paquets d'une même connexion en un enregistrement résumé (durée, volume, protocole), largement utilisée en pratique car elle réduit fortement le volume de données à traiter tout en conservant l'essentiel du signal utile à la détection.
- **Analyse par session ou par entité** : agrège encore davantage, à l'échelle d'un utilisateur ou d'une machine sur une fenêtre de temps (ex. nombre total de connexions échouées sur une heure), ce qui est la granularité typique de l'UEBA.

Le compromis est le suivant : plus la granularité est fine, plus l'information disponible est riche, mais plus le volume de données et le coût de calcul augmentent, et plus l'entraînement du modèle nécessite de données pour éviter le surapprentissage sur des détails non pertinents (chapitre 2).

3. Ingénierie des caractéristiques pour le trafic réseau

Exemple de caractéristiques dérivées classiques (utilisées dans le jeu de données NSL-KDD, réutilisé en TP1) :

- durée de la connexion ;
- type de protocole (TCP, UDP, ICMP) ;
- service ciblé (HTTP, FTP, SSH...) ;
- nombre d'octets source → destination et destination → source ;
- nombre de tentatives de connexion échouées récentes vers le même hôte ;
- taux de connexions vers un même service dans une fenêtre temporelle glissante.

Le passage de paquets réseau bruts à ce type de caractéristiques agrégées est une étape déterminante : un modèle ne peut être meilleur que les caractéristiques qui lui sont fournies.

3.1 Caractéristiques « de contenu » vs caractéristiques « de trafic »

Le jeu NSL-KDD, comme la plupart des jeux de données classiques pour la détection d'intrusion, distingue généralement plusieurs familles de variables, une distinction utile à retenir au-delà du jeu de

données lui-même :

- **Caractéristiques de base (basic features)** : issues directement de l'en-tête de la connexion (durée, protocole, service, drapeaux TCP, volume d'octets).
- **Caractéristiques de contenu (content features)** : dérivées du contenu même de la connexion lorsqu'il est accessible, utiles notamment pour repérer des attaques applicatives (ex. nombre de tentatives de connexion échouées dans une session, indicateurs de commandes suspectes).
- **Caractéristiques de trafic sur fenêtre temporelle (traffic features)** : calculées sur une fenêtre glissante de connexions récentes (par exemple les deux dernières secondes), et particulièrement utiles pour détecter des attaques par balayage (scan) ou par déni de service, qui se manifestent par une répétition rapide de connexions similaires plutôt que par une connexion isolée suspecte.

3.2 Exemple pédagogique d'extraction de caractéristiques

L'extrait suivant illustre, de façon simplifiée, comment calculer une caractéristique de type « fenêtre glissante » (nombre de connexions échouées vers le même hôte dans les deux dernières secondes) à partir d'un journal de connexions déjà horodaté — un exemple pédagogique destiné à illustrer le principe, non un code de production :

```
import pandas as pd

# 'log' : DataFrame fictif avec colonnes ['horodatage', 'hote_dest', 'echec']
log = pd.read_csv("journal_connexions_exemple.csv", parse_dates=["horodatage"])
log = log.sort_values("horodatage").set_index("horodatage")

def compte_echecs_fenetre(sous_journal, hote):
    """Nombre de connexions échouées vers `hote` dans la fenêtre courante."""
    return sous_journal[
        (sous_journal["hote_dest"] == hote) & (sous_journal["echec"] == 1)
    ].shape[0]

# Fenêtre glissante de 2 secondes : pour chaque connexion, on compte
# les échecs récents vers le même hôte, une caractéristique classique
# des jeux de type NSL-KDD/CICIDS.
resultats = []
for horodatage, ligne in log.iterrows():
    fenetre = log.loc[horodatage - pd.Timedelta(seconds=2): horodatage]
    resultats.append(compte_echecs_fenetre(fenetre, ligne["hote_dest"]))

log["echecs_recents_meme_hote"] = resultats
```

Ce type de caractéristique dérivée capture une signature comportementale (une rafale de tentatives échouées) invisible dans une connexion isolée examinée seule, ce qui illustre bien pourquoi l'ingénierie des caractéristiques est souvent plus déterminante pour la performance finale que le choix de l'algorithme lui-même.

4. Approches non supervisées de détection d'anomalies

- **Clustering (K-means, DBSCAN)** : les points isolés ou dans de petits clusters sont considérés suspects.
- **Isolation Forest** : construit des arbres aléatoires ; une observation atypique nécessite en moyenne moins de séparations pour être isolée, ce qui produit un score d'anomalie.
- **Autoencodeurs (réseaux de neurones)** : le modèle apprend à reconstruire les données normales ; une erreur de reconstruction élevée sur une nouvelle observation signale une anomalie potentielle.

4.1 Exemple pédagogique : Isolation Forest avec scikit-learn

```
from sklearn.ensemble import IsolationForest
from sklearn.preprocessing import StandardScaler

# X : matrice de caractéristiques numériques (durée, octets, taux d'erreur, ...)
# déjà extraites selon les principes de la section 3
scaler = StandardScaler()
X_norm = scaler.fit_transform(X)

# contamination : proportion attendue d'anomalies dans les données
# (à ajuster selon le contexte ; ici une valeur pédagogique arbitraire)
detecteur = IsolationForest(
    n_estimators=200, contamination=0.02, random_state=42
)
detecteur.fit(X_norm)

# decision_function : plus le score est faible/négatif, plus
# l'observation est considérée comme anormale
scores_anomalie = detecteur.decision_function(X_norm)
predictions = detecteur.predict(X_norm) # -1 = anomalie, 1 = normal
```

Le paramètre `contamination` mérite une attention particulière : il fixe a priori la proportion attendue d'anomalies dans les données et influence directement le seuil de décision appliqué au score. En contexte réel, cette proportion est rarement connue avec précision — un choix trop faible sous-détecte les attaques, un choix trop élevé génère un excès de fausses alertes. Une pratique courante consiste à traiter le score continu produit par `decision_function` comme un signal de priorisation pour l'analyste plutôt qu'à figer une décision binaire automatique, notamment lors des premiers déploiements d'un tel système.

4.2 Autoencodeurs : intuition et limites

Un autoencodeur est un réseau de neurones entraîné à reproduire en sortie ce qu'il reçoit en entrée, en passant par une couche intermédiaire de dimension réduite (le « goulot d'étranglement »). Entraîné uniquement sur du trafic normal, il apprend une représentation compacte des régularités de ce trafic ; face à une observation qui s'écarte de ces régularités (une attaque), il reconstruit moins bien l'entrée, produisant une **erreur de reconstruction** plus élevée, utilisée comme score d'anomalie.

Cette approche présente un intérêt particulier pour des données de haute dimension ou fortement non linéaires (ex. séquences temporelles de trafic), là où K-means ou Isolation Forest peuvent être moins efficaces. Elle comporte cependant des limites propres :

- Elle nécessite un volume de données d'entraînement nettement plus important que les approches classiques pour converger correctement.
- Le choix de l'architecture (taille du goulot d'étranglement, profondeur du réseau) est déterminant et nécessite une phase d'expérimentation ; un goulot trop large permet au modèle de « tricher » en apprenant presque une fonction identité, réduisant sa capacité de détection.
- Comme tout réseau de neurones profond, elle hérite des enjeux d'explicabilité et de vulnérabilité aux attaques adversariales évoqués au chapitre 6.

5. Approches supervisées

Lorsque des données étiquetées (trafic normal vs attaques connues, ex. jeux NSL-KDD ou CICIDS) sont disponibles, une classification supervisée (Random Forest, SVM, réseaux de neurones) obtient généralement de meilleures performances qu'une approche purement non supervisée, mais reste limitée aux catégories d'attaques représentées dans les données d'entraînement.

5.1 Approches hybrides

En pratique, une architecture fréquemment rencontrée combine les deux paradigmes plutôt que de choisir l'un contre l'autre :

1. Un module **non supervisé** (clustering ou Isolation Forest) filtre en amont un large volume de trafic pour isoler les observations statistiquement atypiques, réduisant considérablement le volume à analyser plus finement.
2. Un module **supervisé**, entraîné sur des catégories d'attaques connues, classe plus précisément les observations retenues (type d'attaque probable), aidant à la priorisation par l'analyste.

Cette architecture en cascade tire parti de la capacité des approches non supervisées à couvrir des menaces inconnues, tout en bénéficiant de la précision et de la richesse d'information (type d'attaque, niveau de confiance) qu'apporte un modèle supervisé sur les catégories déjà documentées.

6. Défis pratiques du déploiement en production

- **Déséquilibre des classes** : les attaques représentent une infime fraction du trafic total ; des techniques de rééquilibrage (sur-échantillonnage, sous-échantillonnage, SMOTE) ou des métriques adaptées (chapitre 2) sont nécessaires.
- **Dérive conceptuelle (concept drift)** : le trafic « normal » évolue dans le temps (nouveaux usages, nouvelles applications), un modèle doit être réentraîné périodiquement.
- **Volumétrie et latence** : un IDS doit souvent traiter des flux en temps réel, ce qui contraint le choix et la complexité du modèle.
- **Intégration avec le SOC** : les alertes générées doivent être exploitables par des analystes humains (contexte, explicabilité, priorisation).

6.1 SMOTE et ses limites

La technique **SMOTE** (*Synthetic Minority Over-sampling Technique*), fréquemment citée pour traiter le déséquilibre des classes, génère de nouveaux exemples synthétiques de la classe minoritaire en interpolant entre des exemples existants proches dans l'espace des caractéristiques, plutôt que de simplement dupliquer les exemples réels (sur-échantillonnage naïf). Bien que largement utilisée, elle présente des limites à connaître avant application :

- Elle peut générer des exemples synthétiques peu réalistes si la classe minoritaire est elle-même composée de sous-groupes très différents (par exemple, plusieurs types d'attaques très distincts regroupés sous une même étiquette « malveillant »), en créant des points intermédiaires qui ne correspondent à aucun comportement d'attaque réel.
- Elle doit impérativement être appliquée **après** la séparation entraînement/test (et recalculée à chaque pli en validation croisée), sous peine de fuite de données : générer des exemples synthétiques avant le découpage risque de placer des points très proches à la fois dans l'ensemble d'entraînement et de test, biaisant l'évaluation.
- Elle ne résout pas le problème de fond si les caractéristiques disponibles ne permettent tout simplement pas de distinguer statistiquement les deux classes.

6.2 Explicabilité opérationnelle : au-delà du score

Un défi pratique souvent sous-estimé est la nécessité de fournir à l'analyste SOC davantage qu'un simple score numérique. Une alerte accompagnée des caractéristiques ayant le plus contribué à la décision (par exemple via l'importance des variables d'une forêt aléatoire, ou des techniques post-hoc comme SHAP pour des modèles plus complexes) permet à l'analyste de valider rapidement la pertinence de l'alerte, d'écarter un faux positif évident, ou de comprendre le vecteur d'attaque probable sans devoir ré-analyser manuellement l'ensemble du trafic concerné. Cette dimension d'**explicabilité opérationnelle** est aussi importante pour l'adoption réelle d'un système par les équipes de sécurité que la performance brute du modèle.

À retenir

- Les systèmes de détection combinent en général signatures (précis mais limités aux menaces connues) et ML par anomalies (couvre potentiellement les menaces nouvelles, au prix de plus de faux positifs).
- La détection par anomalies est particulièrement pertinente contre les attaques ne laissant pas de signature statique exploitable (living-off-the-land, identifiants compromis).
- La qualité des caractéristiques extraites du trafic brut conditionne fortement la performance du modèle ; la granularité d'analyse (paquet, flux, session) est un choix structurant.
- Les architectures hybrides, combinant filtrage non supervisé et classification supervisée en cascade, tirent parti des forces respectives des deux paradigmes.
- Le déséquilibre des classes et la dérive conceptuelle sont deux défis pratiques majeurs, souvent sous-estimés lors du passage du prototype à la production ; des techniques comme SMOTE aident mais ne remplacent pas des caractéristiques de qualité.
- L'explicabilité des alertes auprès des analystes est un facteur clé d'adoption opérationnelle d'un IDS basé ML.

Chapitre 4 — Détection de malwares par machine learning

1. Limites de la détection par signatures pour les malwares

Les antivirus traditionnels reposent largement sur des signatures (empreintes de fichiers malveillants connus). Cette approche est mise en échec par le **polymorphisme** (le malware modifie son code à chaque propagation tout en gardant le même comportement) et le **métamorphisme**, ainsi que par le volume considérable de nouvelles variantes produites quotidiennement, rendant la maintenance d'une base de signatures exhaustive impraticable.

1.1 Signatures « exactes » vs signatures « floues »

Il est utile de distinguer plusieurs générations de signatures pour comprendre pourquoi le ML s'est imposé comme complément :

- **Signatures cryptographiques exactes** (empreinte MD5/SHA-256 d'un fichier connu) : très rapides à vérifier, mais rendues inefficaces par la moindre modification du fichier, même triviale (un seul octet changé produit une empreinte totalement différente).
- **Signatures floues (fuzzy hashing, ex. ssdeep)** : tolèrent de petites variations entre fichiers similaires, offrant une résistance partielle au polymorphisme simple, mais restent limitées face à des transformations plus profondes du code.
- **Règles YARA** : combinent des motifs textuels ou binaires (chaînes de caractères, séquences d'octets) avec une logique booléenne, permettant de couvrir des familles entières de malwares partageant des caractéristiques communes, mais nécessitant une expertise humaine pour être rédigées et maintenues à jour.

Le ML se positionne en complément de ces approches : plutôt que de rechercher une correspondance exacte ou approximative à un motif connu, il apprend à généraliser à partir d'exemples pour reconnaître des caractéristiques *statistiquement* associées aux comportements malveillants, y compris sur des fichiers jamais rencontrés auparavant.

2. Deux approches d'analyse pour l'extraction de caractéristiques

Approche	Principe	Avantage	Limite
Analyse statique	examen du fichier sans l'exécuter (structure du binaire, chaînes de caractères, imports de fonctions système, entropie)	rapide, sans risque d'exécution	contournable par obfuscation, chiffrement du code (packers)

Approche	Principe	Avantage	Limite
Analyse dynamique	exécution du fichier dans un environnement contrôlé (sandbox) et observation du comportement (appels système, connexions réseau, modifications de fichiers/registre)	détecte le comportement réel, résiste mieux à l'obfuscation	plus lente, coûteuse en ressources, certains malwares détectent la sandbox et modifient leur comportement

2.1 Une troisième voie : l'analyse hybride

De nombreux systèmes de détection modernes ne choisissent pas entre analyse statique et dynamique, mais les combinent en une **analyse hybride**, souvent structurée en cascade pour optimiser le compromis rapidité/précision :

1. Une première passe **statique**, rapide et peu coûteuse, filtre la grande majorité des fichiers manifestement sains ou manifestement suspects (score très élevé ou très faible).
2. Seuls les fichiers dans une zone d'incertitude (score intermédiaire) sont soumis à une **analyse dynamique** en sandbox, plus coûteuse mais plus fiable, ce qui limite le volume de fichiers nécessitant cette étape onéreuse.

Cette architecture en cascade illustre un principe général en sécurité opérationnelle : combiner un filtre rapide et peu coûteux avec une analyse approfondie réservée aux cas ambigus, plutôt que d'appliquer uniformément la méthode la plus coûteuse à l'ensemble du flux.

3. Caractéristiques exploitées en analyse statique

- Métadonnées du fichier (taille, en-têtes, section PE pour les exécutables Windows).
- Entropie du fichier ou de ses sections (une entropie élevée peut indiquer un chiffrement/compression, fréquent dans les malwares packés).
- N-grammes d'octets ou d'instructions assembleur.
- Fonctions de l'API système importées (ex. appels réseau, manipulation de la mémoire, techniques d'injection de processus).

3.1 L'entropie, un indicateur simple mais puissant

L'entropie de Shannon mesure le degré de « désordre » ou d'imprévisibilité d'une séquence d'octets. Pour une section de fichier donnée, elle se calcule à partir de la distribution de fréquence des valeurs d'octets (0 à 255) :

$$H = -\sum_{i=0}^{255} p_i \log_2(p_i)$$

où p_i est la fréquence relative de la valeur d'octet i dans la section considérée. Une valeur proche de 8 (le maximum théorique pour des octets sur 8 bits) indique une distribution quasi uniforme des valeurs d'octets, typique de données chiffrées ou fortement compressées ; à l'inverse, du code exécutable ou du texte non compressé présente généralement une entropie plus modérée, car certaines valeurs d'octets (instructions fréquentes, caractères communs) reviennent plus souvent que d'autres.

```

import numpy as np

def entropie_shannon(donnees_octets: bytes) -> float:
    """Calcule l'entropie de Shannon (en bits) d'une séquence d'octets."""
    if len(donnees_octets) == 0:
        return 0.0
    valeurs, comptes = np.unique(
        np.frombuffer(donnees_octets, dtype=np.uint8), return_counts=True
    )
    probabilites = comptes / len(donnees_octets)
    return -np.sum(probabilites * np.log2(probabilites))

# Exemple pédagogique
with open("exemple_binaire.exe", "rb") as f:
    contenu = f.read()

print(f"Entropie globale du fichier : {entropie_shannon(contenu):.2f} bits")

```

En pratique, on calcule souvent l'entropie **section par section** (plutôt que sur le fichier entier) pour repérer une section spécifique anormalement chiffrée au sein d'un exécutable par ailleurs normal — une signature typique d'un *packer* utilisé pour dissimuler le code malveillant réel. L'entropie n'est cependant qu'un indicateur parmi d'autres : de nombreux logiciels légitimes utilisent également compression et chiffrement (mises à jour logicielles, protections anti-piratage), d'où un risque de faux positifs si elle est utilisée isolément.

3.2 N-grammes d'octets : principe et compromis

Un n-gramme d'octets est une sous-séquence contiguë de n octets consécutifs extraite du fichier. En comptant la fréquence de chaque n-gramme observé (souvent pour $n = 2, 3$ ou 4), on construit un vecteur de caractéristiques numériques exploitable par un modèle de classification, de façon conceptuellement proche du bag-of-words utilisé en NLP (chapitre 5) mais appliqué à des octets plutôt qu'à des mots.

```

from collections import Counter

def compte_ngrammes(donnees_octets: bytes, n: int = 3) -> Counter:
    """Compte les n-grammes d'octets consécutifs dans un fichier binaire."""
    return Counter(
        donnees_octets[i:i + n] for i in range(len(donnees_octets) - n + 1)
    )

ngrammes = compte_ngrammes(contenu, n=3)
ngrammes_frequents = ngrammes.most_common(10)

```

Le principal compromis réside dans le choix de n : une valeur faible ($n = 1$ ou 2) capture peu d'information contextuelle mais reste peu coûteuse à calculer ; une valeur plus élevée capture des motifs plus spécifiques (proches de séquences d'instructions réelles) mais fait exploser le nombre de caractéristiques possibles (256^n combinaisons théoriques), nécessitant des techniques de réduction de dimension (sélection des n-grammes les plus discriminants, hachage de caractéristiques) avant d'alimenter un modèle.

4. Caractéristiques exploitées en analyse dynamique (sandbox)

- Séquence d'appels système (syscalls) effectués.
- Connexions réseau initiées (domaines contactés, protocoles).
- Modifications du système de fichiers et de la base de registre.
- Techniques de persistance observées (création de tâches planifiées, clés de démarrage).

4.1 De la séquence brute au vecteur de caractéristiques

Une séquence d'appels système observée en sandbox peut être exploitée de deux façons principales, avec un arbitrage classique entre simplicité et richesse d'information :

- **Approche « sac d'appels » (bag-of-syscalls)** : on compte simplement la fréquence de chaque type d'appel système observé pendant l'exécution, sans tenir compte de leur ordre — une approche simple, analogue au bag-of-words du NLP, mais qui perd toute l'information relative à la séquence des actions (par exemple, l'ordre « ouverture de fichier puis chiffrement puis suppression de l'original », caractéristique d'un ransomware, disparaît si l'on ne compte que les occurrences).
- **Approche séquentielle** : on conserve l'ordre des appels et on utilise un modèle capable de traiter des séquences (chaînes de Markov, réseaux de neurones récurrents, Transformers), capturant ainsi des dépendances temporelles significatives du comportement malveillant, au prix d'une complexité de modélisation plus importante.

4.2 Techniques anti-sandbox employées par les malwares

Les auteurs de malwares connaissent l'existence des sandboxes d'analyse et développent des techniques pour les détecter ou les contourner, ce qui a une conséquence directe sur la fiabilité des caractéristiques dynamiques collectées :

- **Détection de l'environnement virtualisé** : recherche d'artefacts propres aux machines virtuelles (pilotes spécifiques, adresses MAC caractéristiques, ressources matérielles limitées typiques d'un environnement de test).
- **Temporisation (sleep evasion)** : le malware reste inactif pendant une durée dépassant le temps d'observation typique d'une sandbox automatisée, avant de déclencher son comportement réel.
- **Détection d'interaction humaine** : certains malwares vérifient la présence de mouvements de souris, de documents récents ou d'un historique de navigation réaliste avant de s'activer, l'absence de ces signaux suggérant un environnement d'analyse automatisé plutôt qu'un poste utilisateur réel.

Ces techniques expliquent pourquoi un score de « comportement bénin » obtenu en sandbox ne garantit jamais l'innocuité réelle d'un fichier — un principe à retenir pour l'évaluation critique des résultats d'analyse dynamique.

5. Modèles utilisés

- **Classification supervisée** (Random Forest, gradient boosting, réseaux de neurones) sur des caractéristiques extraites statiquement et/ou dynamiquement : approche la plus répandue dans les solutions commerciales (EDR de nouvelle génération).
- **Modèles de séquence** (RNN, Transformers) pour modéliser directement la séquence d'appels système observée en sandbox, capturant mieux les dépendances temporelles du comportement.
- **Clustering** de familles de malwares pour regrouper des variantes proches et prioriser l'analyse humaine.

5.1 Exemple pédagogique : classifieur sur caractéristiques statiques

L'extrait suivant illustre, sur un jeu de caractéristiques statiques fictif, l'entraînement d'un classifieur de gradient boosting — une famille d'algorithmes fréquemment utilisée en pratique pour ce type de problème car elle combine bonne performance et robustesse aux variables hétérogènes (numériques et catégorielles mélangées) :

```
from sklearn.ensemble import GradientBoostingClassifier
from sklearn.model_selection import cross_val_score

# X : caractéristiques statiques (entropie par section, taille,
# nombre de fonctions importées suspectes, présence d'un packer connu, ...)
# y : 1 = malveillant, 0 = sain (exemple pédagogique)
modele = GradientBoostingClassifier(
    n_estimators=300, max_depth=4, learning_rate=0.05, random_state=42
)

scores = cross_val_score(modele, X, y, cv=5, scoring="f1")
print(f"F1-score moyen (validation croisée) : {scores.mean():.3f}")

modele.fit(X, y)

# Importance des variables : utile pour comprendre quelles
# caractéristiques statiques pèsent le plus dans la décision
importances = sorted(
    zip(X.columns, modele.feature_importances_), key=lambda t: -t[1]
)
print("Caractéristiques les plus discriminantes :", importances[:5])
```

L'inspection de l'importance des variables (`feature_importances_`) est particulièrement utile en détection de malwares : elle permet de vérifier, avant tout déploiement, que le modèle s'appuie bien sur des caractéristiques légitimement liées au comportement malveillant, et non sur un artefact fortuit du jeu de données d'entraînement (par exemple, une caractéristique corrélée par hasard avec la source d'où proviennent les échantillons collectés, plutôt qu'avec leur nature réellement malveillante).

6. Techniques d'évasion contre les détecteurs ML

- **Obfuscation et packing** : rend l'analyse statique moins efficace en masquant les caractéristiques exploitables.
- **Détection d'environnement virtualisé/sandbox** : le malware modifie son comportement (ou reste inactif) s'il détecte qu'il s'exécute dans un environnement d'analyse.

- **Exemples adversariaux** : modification minimale d'un binaire (ajout d'octets inoffensifs, réorganisation de sections) pour franchir la frontière de décision d'un modèle sans changer le comportement malveillant réel — approfondi au chapitre 6.

6.1 Pourquoi les exemples adversariaux sont particulièrement efficaces contre les modèles statiques

Une caractéristique propre au domaine des malwares (contrairement, par exemple, à la classification d'images) est l'existence de nombreuses transformations qui modifient les caractéristiques statiques extraites **sans modifier le comportement fonctionnel** du fichier : ajout d'octets inutilisés en fin de fichier, insertion de sections vides, réordonnancement de sections non référencées par le code, ajout de chaînes de caractères inoffensives. Ces transformations peuvent suffire à déplacer un fichier au-delà de la frontière de décision d'un modèle statique entraîné sur des caractéristiques superficielles (taille, entropie globale, présence de certaines chaînes), sans que le fichier cesse d'être malveillant à l'exécution.

Ce constat a une conséquence pédagogique importante : un modèle de détection statique performant sur un jeu de test doit être évalué non seulement sur sa performance brute, mais aussi sur sa **robustesse** face à ce type de manipulation triviale — une dimension d'évaluation trop souvent négligée dans les démonstrations commerciales de solutions de détection.

7. Limites et bonnes pratiques

Un détecteur ML de malwares doit être régulièrement réentraîné (nouvelles familles), évalué sur des échantillons réellement représentatifs (pas seulement des échantillons « faciles »), et combiné avec des mécanismes complémentaires (réputation de fichiers, analyse comportementale en production, threat intelligence) plutôt qu'utilisé comme unique ligne de défense.

7.1 Le biais de collecte des échantillons d'entraînement

Un piège méthodologique fréquent dans la constitution d'un jeu de données de malwares mérite une attention particulière : les échantillons « bénins » et « malveillants » proviennent souvent de sources très différentes (par exemple, des logiciels sains téléchargés depuis un dépôt officiel d'un côté, des échantillons malveillants soumis à un service d'analyse en ligne de l'autre). Un modèle entraîné sur un tel jeu de données risque d'apprendre à distinguer la *source de collecte* plutôt que la nature réellement malveillante du fichier (par exemple, des différences systématiques de format de compilation, de signature numérique, ou de métadonnées propres à chaque source) — un phénomène apparenté au **raccourci d'apprentissage (shortcut learning)** documenté dans d'autres domaines du machine learning. Une performance apparemment excellente en test peut ainsi masquer une généralisation très faible en conditions réelles.

La bonne pratique consiste à constituer, autant que possible, des échantillons bénins et malveillants issus de contextes de collecte comparables, et à évaluer systématiquement le modèle sur des familles de malwares et des sources absentes de l'ensemble d'entraînement, pour estimer sa capacité de généralisation réelle plutôt que sa capacité à reconnaître les biais du jeu de données.

À retenir

- L'analyse statique est rapide mais contournable par obfuscation ; l'analyse dynamique observe le comportement réel mais coûte plus cher et peut être détectée par le malware ; l'analyse hybride en cascade combine les deux pour optimiser le compromis coût/précision.
- L'entropie et les n-grammes d'octets sont des caractéristiques statiques simples mais puissantes ; leur interprétation nécessite néanmoins prudence (compression légitime, choix du paramètre n).
- Les malwares emploient des techniques actives de détection de sandbox, ce qui limite la fiabilité d'un verdict « bénin » obtenu uniquement par analyse dynamique automatisée.
- Les modèles de classification supervisée sur caractéristiques extraites sont l'approche la plus répandue en pratique ; l'inspection de l'importance des variables aide à détecter un apprentissage fondé sur des artefacts non pertinents du jeu de données.
- Les techniques d'évasion (obfuscation, détection de sandbox, exemples adversariaux) rappellent qu'un détecteur ML n'est jamais une solution unique et définitive.
- Le biais de collecte entre échantillons sains et malveillants est un piège méthodologique fréquent, pouvant conduire un modèle à apprendre la source des données plutôt que leur nature réellement malveillante.

Chapitre 5 — NLP appliqué à la sécurité (phishing, analyse de logs)

1. Le traitement automatique du langage naturel (NLP) en cybersécurité

Le NLP permet d'analyser automatiquement des contenus textuels : e-mails, messages, URL, journaux d'événements, rapports de threat intelligence. En sécurité, ses deux applications les plus répandues sont la détection de phishing et l'analyse assistée de journaux volumineux.

1.1 Pourquoi le texte est un signal particulièrement riche mais difficile

Le texte présente une combinaison de propriétés qui le rend à la fois précieux et délicat à exploiter pour la sécurité :

- **Richesse sémantique** : un message de phishing peut être identifié par son intention (créer un sentiment d'urgence, inciter à une action) bien au-delà de simples mots-clés, ce qu'aucune règle statique simple ne peut capturer exhaustivement.
- **Variabilité et créativité linguistique** : contrairement à un flux réseau structuré (chapitre 3) ou à un binaire (chapitre 4), le langage naturel offre une infinité de façons d'exprimer la même intention, ce qui complique la détection par simples règles ou motifs.
- **Manipulation délibérée par l'attaquant** : un attaquant en phishing adapte activement son texte pour éviter les filtres, un phénomène de « jeu du chat et de la souris » qui rapproche ce domaine de la détection de malwares (chapitre 4) plus que d'applications NLP classiques (analyse de sentiment sur des avis clients, par exemple), où le texte analysé n'a pas d'intention adverse.

2. Représentation du texte pour le machine learning

Un modèle ML ne manipule pas directement du texte : il faut le transformer en représentation numérique.

Méthode	Principe	Caractéristique
Bag-of-words / TF-IDF	compte (pondéré) l'occurrence des mots, sans tenir compte de l'ordre	simple, efficace pour des tâches de classification classiques
N-grammes de caractères	séquences de n caractères consécutifs	robuste aux fautes d'orthographe volontaires (technique d'évasion courante en phishing)
Word embeddings (Word2Vec, GloVe)	représentation vectorielle dense capturant des relations sémantiques entre mots	capture le sens, plus coûteux à entraîner

Méthode	Principe	Caractéristique
Modèles de langage pré-entraînés (Transformers, type BERT)	représentation contextuelle, état de l'art actuel	performance élevée, coût de calcul plus important

2.1 TF-IDF en détail

Le TF-IDF (*Term Frequency – Inverse Document Frequency*) pondère chaque mot d'un document selon deux composantes complémentaires :

- **TF (fréquence du terme)** : plus un mot apparaît fréquemment dans un document donné, plus il est susceptible d'être significatif pour ce document.
- **IDF (fréquence inverse de document)** : un mot apparaissant dans presque tous les documents du corpus (comme les mots grammaticaux très courants) apporte peu d'information discriminante et voit son poids réduit ; à l'inverse, un mot rare mais présent dans peu de documents est jugé plus informatif.

Formellement, pour un terme t et un document d appartenant à un corpus de N documents :

$$\text{TF-IDF}(t, d) = \text{TF}(t, d) \times \log\left(\frac{N}{\text{DF}(t)}\right)$$

où $\text{DF}(t)$ est le nombre de documents du corpus contenant le terme t . Cette pondération explique pourquoi TF-IDF reste une base de référence solide en classification de texte malgré son ancienneté : elle capture, avec un coût de calcul minime, l'idée intuitive qu'un mot rare et répété dans un message donné (par exemple, un nom de marque usurpée répété plusieurs fois dans un e-mail de phishing) est plus significatif qu'un mot omniprésent dans tout le corpus.

```

from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.linear_model import LogisticRegression
from sklearn.pipeline import make_pipeline

# corpus_emails : liste de textes d'e-mails (exemple pédagogique)
# labels : 1 = phishing, 0 = légitime
vectoriseur = TfidfVectorizer(
    lowercase=True,
    ngram_range=(1, 2),    # unigrammes et bigrammes de mots
    max_features=20000,    # limite le vocabulaire aux termes les plus fréquents
    stop_words=None,       # attention : ne pas retirer aveuglément les mots
                          # vides en phishing, certains (« urgent », « cliquez »)
                          # sont eux-mêmes des indicateurs utiles
)

pipeline = make_pipeline(vectoriseur, LogisticRegression(max_iter=1000))
pipeline.fit(corpus_emails, labels)

# Inspection des mots les plus associés à la classe "phishing"
import numpy as np
vocab = vectoriseur.get_feature_names_out()
poids = pipeline.named_steps["logisticregression"].coef_[0]
top_indices = np.argsort(poids)[-15:]
print("Termes les plus associés au phishing :", vocab[top_indices])

```

Une remarque pédagogique importante : dans les tâches NLP « génériques », il est courant de retirer les **mots vides** (*stop words* : « le », « de », « et »...), considérés comme peu informatifs. En détection de phishing, cette pratique doit être appliquée avec prudence : certains mots très fréquents (« urgent », « cliquez », « vérifier », « suspendu ») sont au contraire des indicateurs discriminants forts, et les retirer par un filtre générique de mots vides risquerait de supprimer un signal utile.

2.2 N-grammes de caractères : robustesse à l'évasion orthographique

Une limite du bag-of-words au niveau du mot est sa fragilité face à de simples variations orthographiques volontaires — une technique d'évasion très répandue en phishing et en spam (par exemple remplacer « virement » par « vlirement » ou insérer des espaces ou caractères invisibles au milieu d'un mot-clé filtré). Les n-grammes de *caractères* (plutôt que de mots) offrent une robustesse partielle à ce type de manipulation, car une grande partie des sous-séquences de caractères reste inchangée malgré la modification :

```

from sklearn.feature_extraction.text import TfidfVectorizer

vectoriseur_car = TfidfVectorizer(
    analyzer="char_wb",    # n-grammes de caractères, respectant les limites de mots
    ngram_range=(3, 5),
    max_features=20000,
)

```

Ce type de représentation est également largement utilisé pour la détection d'URL malveillantes (section 4), où l'unité « mot » est souvent moins pertinente que les sous-chaînes de caractères composant le nom de domaine.

2.3 Des embeddings statiques aux Transformers : ce que le contexte apporte

Les méthodes de type Word2Vec ou GloVe associent à chaque mot un vecteur **fixe**, indépendant du contexte dans lequel il apparaît : le mot « virus » aura toujours le même vecteur, qu'il apparaisse dans un contexte informatique ou médical. Les modèles de langage pré-entraînés de type Transformer (BERT et ses variantes) produisent au contraire une représentation **contextuelle** : le vecteur associé à un mot dépend des mots qui l'entourent dans la phrase, ce qui permet de désambiguïser le sens et de capturer des relations syntaxiques et sémantiques plus fines (négation, ironie, structure de phrase).

Ce gain de finesse a un coût : l'entraînement (ou même le simple usage en inférence) de ces modèles est nettement plus coûteux en calcul que TF-IDF ou les embeddings statiques, ce qui doit être mis en balance avec les contraintes opérationnelles (latence, volume de messages à traiter en temps réel) d'un système de filtrage de messagerie à grande échelle. En pratique, il est fréquent de partir d'un modèle pré-entraîné sur un vaste corpus généraliste, puis de l'affiner (*fine-tuning*) sur un corpus spécifique à la tâche de détection de phishing — une approche de **transfert d'apprentissage** qui réduit fortement le volume de données étiquetées nécessaire par rapport à un entraînement entièrement from scratch.

3. Détection de phishing par e-mail

Caractéristiques exploitées, combinant NLP et méta-données :

- contenu textuel du message (urgence, demande d'action, fautes, incitation au clic) ;
- caractéristiques de l'expéditeur (domaine récemment enregistré, usurpation d'affichage — *display name spoofing*) ;
- caractéristiques des URL contenues (longueur, présence de caractères trompeurs, similarité avec un domaine légitime — *typosquatting*) ;
- en-têtes techniques du message (résultats SPF/DKIM/DMARC, incohérences de routage).

Une approche efficace combine généralement un modèle NLP sur le contenu du message et des règles/caractéristiques dédiées à l'analyse des URL et des en-têtes, plutôt qu'un seul modèle « texte brut ».

3.1 Pourquoi une approche multi-signaux surpasse un modèle « texte seul »

Cette combinaison n'est pas un simple choix d'ingénierie : elle répond à une limite structurelle du texte seul comme signal de détection. Un attaquant sophistiqué peut produire un texte irréprochable, sans faute et parfaitement crédible (d'autant plus facilement aujourd'hui avec l'assistance de modèles de langage génératifs, voir section 6), rendant les indicateurs purement textuels insuffisants. À l'inverse, les caractéristiques techniques — authenticité du domaine expéditeur, cohérence des enregistrements SPF/DKIM/DMARC, âge d'enregistrement du nom de domaine — sont beaucoup plus difficiles à falsifier pour un attaquant, car elles dépendent d'une infrastructure qu'il ne contrôle pas entièrement (registres de noms de domaine, autorités de certification, réputation historique).

Un système robuste combine ainsi typiquement plusieurs modèles ou caractéristiques en un **score composite** :

```
# Exemple pédagogique simplifié d'agrégation de plusieurs signaux
# en un score de risque de phishing (les poids sont illustratifs)

def score_risque_phishing(email):
    score_texte = modele_nlp.predict_proba([email.corps])[0][1]          # 0 à 1
    score_url = modele_url.predict_proba([email.urls_extraites])[0][1]   # 0 à 1
    domaine_recent = 1.0 if email.age_domaine_jours < 30 else 0.0
    auth_echouee = 1.0 if not email.spf_dkim_dmarc_ok else 0.0

    score_final = (
        0.4 * score_texte
        + 0.3 * score_url
        + 0.15 * domaine_recent
        + 0.15 * auth_echouee
    )
    return score_final
```

Cet exemple, volontairement simplifié (les poids réels seraient déterminés par apprentissage plutôt que fixés arbitrairement, par exemple via un modèle de stacking combinant les scores des sous-modèles comme nouvelles caractéristiques d'un modèle final), illustre le principe de défense en profondeur appliqué au filtrage anti-phishing : aucun signal isolé n'est fiable à lui seul, mais leur combinaison réduit fortement les angles morts de chaque approche prise individuellement.

4. Détection d'URL malveillantes

Approche fréquente : classification supervisée à partir de caractéristiques lexicales de l'URL elle-même (longueur, nombre de sous-domaines, présence de caractères spéciaux, entropie), sans nécessairement visiter la page cible — utile pour un filtrage rapide en amont.

4.1 Catégories de caractéristiques d'URL

Au-delà des caractéristiques purement lexicales mentionnées, on distingue généralement plusieurs familles de caractéristiques exploitées en détection d'URL, chacune capturant un aspect différent du risque :

- **Caractéristiques lexicales** : longueur totale de l'URL, nombre de sous-domaines, présence de caractères inhabituels (@, tirets multiples), usage d'une adresse IP brute à la place d'un nom de domaine, entropie de la chaîne de caractères.
- **Caractéristiques de réputation du domaine** : ancienneté d'enregistrement (les domaines de phishing sont très souvent enregistrés peu avant la campagne d'attaque), historique de la réputation du domaine, présence dans des listes de réputation connues.
- **Caractéristiques de similarité (typosquatting)** : distance d'édition (par exemple la distance de Levenshtein) entre le domaine observé et des domaines légitimes connus et fréquemment ciblés, permettant de repérer des variantes comme un « o » remplacé par un « 0 », ou l'insertion d'un caractère supplémentaire.
- **Caractéristiques de contenu de la page** (lorsque la page est effectivement visitée, ce qui n'est pas toujours souhaitable pour des raisons de sécurité et de latence) : présence d'un formulaire de connexion imitant une marque connue, ressemblance visuelle avec un site légitime.

```

import numpy as np

def distance_levenshtein(a: str, b: str) -> int:
    """Distance d'édition simple entre deux chaînes (implémentation pédagogique)."""
    m, n = len(a), len(b)
    d = np.zeros((m + 1, n + 1), dtype=int)
    d[:, 0] = np.arange(m + 1)
    d[0, :] = np.arange(n + 1)
    for i in range(1, m + 1):
        for j in range(1, n + 1):
            cout = 0 if a[i - 1] == b[j - 1] else 1
            d[i, j] = min(d[i - 1, j] + 1, d[i, j - 1] + 1, d[i - 1, j - 1] + cout)
    return d[m, n]

# Exemple : repérer un domaine potentiellement usurpateur
domaines_legitimes = ["banque-exemple.com", "monservice.org"]
domaine_observe = "banque-exempl3.com"

distances = [distance_levenshtein(domaine_observe, d) for d in domaines_legitimes]
print("Distances aux domaines légitimes connus :", distances)

```

Une distance d'édition faible (par exemple 1 ou 2) entre un domaine observé et un domaine légitime largement connu, combinée à une faible ancienneté d'enregistrement, constitue un indicateur de typosquatting particulièrement fiable, bien plus robuste qu'une analyse purement lexicale de l'URL prise isolément.

5. Analyse de journaux (log analysis) assistée par NLP

Les journaux systèmes et applicatifs peuvent être traités comme du texte semi-structuré :

- **Regroupement de motifs (log clustering / log parsing)** : regrouper des lignes de log similaires en « templates » (ex. outils comme Drain), pour réduire des millions de lignes à quelques centaines de motifs analysables.
- **Détection d'anomalies séquentielles** : modéliser la séquence normale d'événements (via un modèle de séquence) et détecter les déviations, potentiellement révélatrices d'un comportement d'attaque (ex. séquence anormale de commandes après une compromission).
- **Résumé automatique** : synthétiser de grands volumes d'alertes pour un analyste SOC, en s'appuyant de plus en plus sur des modèles de langage génératifs (avec vigilance sur la fiabilité de leurs sorties, cf. chapitre 6).

5.1 Principe du parsing de logs (illustration simplifiée)

Une ligne de log brute comme Connexion échouée pour l'utilisateur alice depuis 10.0.0.5 et une autre Connexion échouée pour l'utilisateur bob depuis 10.0.0.9 partagent un même **template** sous-jacent (Connexion échouée pour l'utilisateur <VAR> depuis <VAR>), les parties variables correspondant aux paramètres spécifiques de chaque événement. Les outils de parsing de logs (dont l'algorithme Drain, largement cité dans la littérature) automatisent l'extraction de ces templates à partir d'un flux brut de journaux, permettant de transformer un volume considérable de lignes en un nombre restreint de motifs, sur lesquels des techniques classiques (comptage de

fréquence, détection d'anomalies sur la distribution des templates au cours du temps) deviennent applicables :

```
import re

def extraction_template_simplifiee(ligne_log: str) -> str:
    """
    Illustration très simplifiée du principe de templating :
    remplace les nombres et adresses IP par un marqueur générique.
    (Une implémentation réelle, type Drain, est nettement plus robuste
    et s'appuie sur un arbre de recherche par préfixes.)
    """
    ligne = re.sub(r"\b\d{1,3}(\.\d{1,3}){3}\b", "<IP>", ligne_log)
    ligne = re.sub(r"\b\d+\b", "<NUM>", ligne)
    return ligne

exemples = [
    "Connexion échouée pour l'utilisateur alice depuis 10.0.0.5",
    "Connexion échouée pour l'utilisateur bob depuis 10.0.0.9",
]
for ligne in exemples:
    print(extraction_template_simplifiee(ligne))
```

Une fois les journaux réduits à une séquence de templates, on peut appliquer les techniques de détection d'anomalies séquentielles vues au chapitre 3 (modélisation de la séquence normale d'événements, alerte sur les déviations), ou simplement surveiller l'apparition d'un template rare ou inédit — une piste simple mais souvent efficace pour détecter des événements systèmes inhabituels sans construire un modèle complexe.

5.2 Prudence sur l'usage de modèles génératifs pour le résumé d'alertes

L'usage croissant de modèles de langage génératifs pour synthétiser de grands volumes d'alertes à destination d'un analyste SOC (par exemple, produire un résumé en langage naturel d'une série d'événements corrélés) offre un gain de productivité réel, mais introduit un risque spécifique : ces modèles peuvent produire des affirmations plausibles mais factuellement incorrectes (« hallucinations »), en particulier lorsqu'ils sont sollicités pour interpréter des événements ambigus ou incomplets. Un résumé généré automatiquement doit donc rester **vérifiable** (idéalement accompagné des événements bruts sources) et ne jamais se substituer entièrement au jugement de l'analyste pour une décision de sécurité critique — ce point est approfondi au chapitre 6 sous l'angle plus large de la fiabilité des systèmes d'IA générative.

6. Limites spécifiques au NLP en sécurité

- Les attaquants adaptent activement leur texte pour éviter la détection (fautes volontaires, homoglyphes, texte caché dans des images) — un jeu du chat et de la souris comparable à celui vu pour les malwares.
- Un modèle NLP entraîné sur une langue ou un contexte culturel donné généralise mal à d'autres langues ou contextes.

- Le contenu généré automatiquement par des modèles de langage (utilisé de façon offensive par des attaquants pour rédiger des messages de phishing très convaincants et sans fautes, y compris en plusieurs langues) réduit l'efficacité des indicateurs textuels classiques (fautes d'orthographe, formulations maladroites) longtemps utilisés pour repérer le phishing.

6.1 Les homoglyphes : une évasion visuelle difficile à détecter par le texte seul

Les attaques par **homoglyphes** exploitent la ressemblance visuelle entre caractères provenant de jeux de caractères différents (par exemple, la lettre latine « a » et une lettre cyrillique visuellement quasi identique) pour construire des noms de domaine ou des mots qui semblent identiques à l'œil humain mais différent au niveau des caractères Unicode réellement utilisés. Un filtre NLP opérant uniquement sur la représentation textuelle brute peut être trompé si les deux caractères sont traités comme complètement distincts par le modèle de représentation, alors qu'un humain ne perçoit aucune différence. Des contre-mesures existent (normalisation Unicode, détection de scripts mixtes dans un même nom de domaine — un domaine légitime mélange rarement plusieurs alphabets), mais illustrent bien que la robustesse d'un système NLP de sécurité doit être pensée au niveau de l'encodage des caractères, pas seulement au niveau du sens des mots.

À retenir

- Le NLP transforme du texte en représentation numérique exploitable par un modèle (TF-IDF, n-grammes de caractères, embeddings, Transformers), avec un compromis performance/coût de calcul et de robustesse à l'évasion.
- TF-IDF reste une base de référence solide et peu coûteuse ; les n-grammes de caractères apportent une robustesse particulière aux variations orthographiques volontaires, fréquentes en phishing.
- La détection de phishing combine efficacement analyse du contenu textuel et caractéristiques techniques (URL, en-têtes, authentification du domaine, ancienneté du nom de domaine), car le texte seul devient insuffisant face à des messages sans fautes générés ou peaufinés par IA.
- L'analyse de journaux par NLP passe souvent par une étape de regroupement en templates (log parsing), qui réduit drastiquement le volume à traiter avant d'appliquer des techniques de détection d'anomalies séquentielles.
- L'essor des modèles de langage génératifs facilite la production de contenus de phishing convaincants, ce qui affaiblit les indicateurs textuels traditionnels et pousse vers des défenses combinant plusieurs signaux ; ces mêmes modèles, utilisés côté défense pour résumer des alertes, doivent être employés avec prudence en raison du risque d'hallucination.

Chapitre 6 — Sécurité de l'IA : attaques adversariales et IA offensive

1. L'IA comme nouvelle surface d'attaque

À mesure que des systèmes d'IA sont déployés dans des fonctions sensibles (détection de fraude, contrôle d'accès biométrique, filtrage de contenu, systèmes autonomes), ils deviennent eux-mêmes des cibles. Le référentiel **OWASP Machine Learning Security Top 10** et le cadre **MITRE ATLAS** recensent ces menaces spécifiques.

1.1 Pourquoi la sécurité de l'IA ne se réduit pas à la sécurité logicielle classique

Un système ML introduit des surfaces d'attaque qui n'ont pas d'équivalent direct dans la sécurité logicielle traditionnelle, ce qui justifie de le traiter comme une discipline à part entière plutôt que comme un simple cas particulier de sécurité applicative :

- **La logique du système n'est pas écrite explicitement par un développeur**, mais apprise à partir de données. Il n'existe donc pas de code source à auditer ligne par ligne pour garantir un comportement correct : la « logique » est distribuée dans des millions ou des milliards de paramètres numériques, ce qui rend l'audit traditionnel (revue de code, analyse statique) largement inapplicable.
- **Les données d'entraînement deviennent elles-mêmes une surface d'attaque** (section 3), alors qu'en sécurité logicielle classique, les données de configuration ne sont généralement pas le vecteur principal de compromission du comportement du programme.
- **Le modèle entraîné peut être considéré comme un actif à protéger en tant que tel** (propriété intellectuelle, avantage concurrentiel), au même titre que le code source d'un logiciel classique, mais avec des vecteurs de vol différents (extraction par requêtes successives, section 4).
- **La frontière entre comportement « normal » et comportement « attaqué »** peut être ténue et dépendre de l'interprétation du contexte (une entrée légèrement modifiée reste-t-elle une entrée légitime « bruitée » ou une attaque délibérée ?), contrairement à une vulnérabilité logicielle classique où l'exploitation produit généralement un comportement clairement anormal (crash, exécution de code arbitraire).

Ces particularités expliquent l'émergence de cadres dédiés comme MITRE ATLAS (*Adversarial Threat Landscape for Artificial-Intelligence Systems*), qui adapte au domaine de l'IA une structuration en tactiques et techniques inspirée du cadre MITRE ATT&CK bien connu en sécurité classique, tout en introduisant des catégories propres au ML (empoisonnement, exfiltration de modèle, évasion) absentes du cadre original.

2. Attaques adversariales (evasion attacks)

Principe : modifier légèrement une entrée, de façon souvent imperceptible pour un humain, pour tromper un modèle et lui faire produire une prédiction erronée, sans modifier le comportement réel de l'objet analysé.

- **Sur des images** : ajouter un bruit calculé, invisible à l'œil nu, qui fait classer une image de panneau stop comme un panneau de limitation de vitesse par un modèle de vision.
- **Sur des malwares** : ajouter des octets inoffensifs à un binaire malveillant pour franchir la frontière de décision d'un classifieur, sans changer le comportement malveillant réel (évoqué au chapitre 4).
- **Sur du texte (phishing, spam)** : introduire des variations orthographiques, des caractères invisibles ou des synonymes pour échapper à un filtre NLP tout en restant compréhensible par un humain.

2.1 Intuition géométrique : pourquoi une perturbation imperceptible suffit

Un modèle de classification définit implicitement une **frontière de décision** dans l'espace des caractéristiques d'entrée, séparant les régions associées à chaque classe prédite. Une observation légitime se situe généralement loin de cette frontière, dans une région où le modèle est « confiant ». Une attaque adversariale cherche la perturbation de **plus petite amplitude possible** (au sens d'une distance donnée, par exemple la norme euclidienne pour une image) qui déplace le point représentant l'entrée juste de l'autre côté de la frontière de décision.

Ce qui rend ces attaques particulièrement redoutables sur des modèles de deep learning tient à une propriété géométrique associée à la haute dimension de l'espace d'entrée (des millions de pixels pour une image, par exemple) : dans un espace de très grande dimension, il existe généralement une direction de perturbation qui déplace fortement la prédiction du modèle tout en modifiant très peu chaque composante individuelle de l'entrée (chaque pixel), ce qui rend la perturbation totale imperceptible pour un observateur humain tout en étant efficace du point de vue du modèle. Cette observation, documentée dès les premiers travaux sur le sujet (par exemple la méthode *Fast Gradient Sign Method*, qui exploite directement le gradient du modèle pour calculer une perturbation efficace en une seule étape), a durablement structuré la recherche en robustesse adversariale du deep learning.

Modèles d'attaque

Modèle	Connaissance de l'attaquant
Boîte blanche (white-box)	accès complet au modèle (architecture, paramètres) — permet de calculer des perturbations optimales
Boîte noire (black-box)	accès uniquement aux prédictions du modèle (requêtes successives) — attaques par substitution (entraîner un modèle de substitution pour approximer les frontières de décision)

2.2 Transférabilité : pourquoi le boîte noire n'est pas rassurant

Une propriété empirique importante, souvent sous-estimée, réduit fortement l'intérêt pratique de la distinction boîte blanche/boîte noire du point de vue défensif : la **transférabilité des exemples**

adversariaux. Un exemple adversarial conçu pour tromper un modèle donné a souvent de bonnes chances de tromper également un *autre* modèle entraîné sur une tâche similaire, même avec une architecture différente. Cette propriété permet à un attaquant en situation de boîte noire de contourner l'absence d'accès direct au modèle cible : il entraîne un **modèle de substitution** sur des données similaires (ou simplement à partir des prédictions obtenues en interrogeant le modèle cible), conçoit une attaque en boîte blanche contre ce modèle de substitution, puis applique la perturbation obtenue au modèle cible réel, en espérant qu'elle transfère avec succès.

Cette propriété a une implication défensive directe : « cacher » les détails d'un modèle (une approche parfois appelée *security through obscurity*) offre une protection bien plus limitée qu'il n'y paraît intuitivement contre les attaques adversariales, et ne doit jamais constituer la seule ligne de défense.

3. Empoisonnement de données (data poisoning)

Attaque visant la **phase d'entraînement** plutôt que l'inférence : un attaquant capable d'injecter des données malveillantes dans le jeu d'entraînement (ex. un système qui se ré-entraîne en continu sur des données utilisateurs) peut dégrader la performance générale du modèle ou y insérer une **porte dérobée (backdoor)** : le modèle se comporte normalement sauf en présence d'un déclencheur spécifique connu de l'attaquant, qui provoque alors une prédiction choisie.

3.1 Empoisonnement « en aveugle » vs empoisonnement ciblé

Il est utile de distinguer deux familles d'objectifs pour un attaquant menant une attaque d'empoisonnement, car elles appellent des contre-mesures différentes :

- **Empoisonnement de disponibilité (availability poisoning)** : l'attaquant vise simplement à dégrader la performance générale du modèle (le rendre globalement moins fiable), par exemple en injectant un volume significatif de données mal étiquetées ou bruitées. Cet objectif est relativement facile à détecter *a posteriori*, car il produit une dégradation mesurable et globale de la performance sur un jeu de validation propre.
- **Empoisonnement ciblé avec porte dérobée (backdoor / targeted poisoning)** : l'attaquant vise à faire échouer le modèle uniquement sur des entrées spécifiques (contenant un déclencheur particulier), tout en préservant une performance normale, voire excellente, sur toutes les autres entrées. Ce type d'attaque est nettement plus difficile à détecter, car le modèle « empoisonné » réussit la quasi-totalité des tests d'évaluation standards — seul un test ciblé sur des entrées contenant le déclencheur révélerait le problème, ce qui suppose de savoir *a priori* qu'un tel déclencheur pourrait exister.

Un exemple pédagogique illustrant l'intuition d'une porte dérobée : un système de détection de spam ré-entraîné périodiquement sur les messages que les utilisateurs signalent ou ne signalent pas pourrait être « empoisonné » si un attaquant parvient, à grande échelle, à faire en sorte que des messages contenant un mot-clé anodin spécifique soient systématiquement traités comme légitimes par un grand nombre de comptes compromis ou complices — le modèle apprendrait alors, sans intention explicite de ses concepteurs, à associer ce mot-clé à un comportement « non spam », que l'attaquant pourrait ensuite exploiter dans ses propres campagnes.

3.2 Facteurs de risque et contre-mesures pratiques

Le risque d'empoisonnement dépend fortement du **mode d'entraînement** du système :

- Un modèle entraîné **une fois hors ligne**, sur un jeu de données figé et audité, est nettement moins exposé qu'un système en **apprentissage continu (online learning)**, qui intègre en permanence de nouvelles données provenant potentiellement d'utilisateurs non fiables.
- Le degré de contrôle sur la **provenance** des données d'entraînement (données internes auditées vs données collectées automatiquement sur des sources ouvertes ou semi-ouvertes) est un facteur de risque déterminant.

Les contre-mesures pratiques incluent la validation statistique des nouvelles données avant intégration (détection d'anomalies sur les données elles-mêmes, avant même d'entraîner le modèle dessus), la traçabilité des sources (savoir d'où provient chaque exemple d'entraînement), la limitation du poids d'une source unique dans le jeu de données global, et des tests de robustesse ciblés recherchant explicitement des comportements de porte dérobée avant tout déploiement en production.

4. Extraction et inversion de modèle

- **Extraction de modèle (model extraction)** : reconstruire une approximation d'un modèle propriétaire à partir de requêtes successives à son API, permettant de le voler ou de préparer une attaque adversariale en boîte blanche sur la copie.
- **Inversion de modèle / attaque par appartenance (membership inference)** : déterminer si une donnée spécifique a été utilisée dans l'entraînement d'un modèle, ou reconstruire partiellement des données d'entraînement sensibles — un enjeu direct de confidentialité (RGPD) lorsque le modèle a été entraîné sur des données personnelles.

4.1 Pourquoi l'attaque par appartenance fonctionne : le lien avec le surapprentissage

L'attaque par appartenance (*membership inference*) exploite une observation liée directement à une notion vue au chapitre 2 : un modèle qui a **surappris** (overfitting) se comporte différemment sur les données qu'il a vues à l'entraînement par rapport à des données nouvelles — typiquement, il produit des prédictions avec une confiance plus élevée (ou une erreur plus faible) sur les exemples d'entraînement mémorisés que sur des exemples jamais vus, même de même nature. Un attaquant disposant d'un accès aux scores de confiance produits par le modèle (et non seulement à la décision finale) peut exploiter cet écart statistique pour deviner, avec une fiabilité supérieure au hasard, si une donnée spécifique a fait partie de l'ensemble d'entraînement.

Cette observation a une conséquence défensive concrète : les techniques de régularisation qui limitent le surapprentissage (évoquées au chapitre 2 comme bonnes pratiques de généralisation) réduisent également, en tant qu'effet secondaire bénéfique, la vulnérabilité du modèle à ce type d'attaque de confidentialité — un des rares cas où bonne pratique de performance et bonne pratique de sécurité coïncident directement.

4.2 Confidentialité différentielle : une garantie mathématique plutôt qu'une simple bonne pratique

Contrairement à des mesures ad hoc, la **confidentialité différentielle (differential privacy)** offre une garantie mathématique formelle : elle consiste à ajouter un bruit calibré, soit aux données d'entraînement, soit au processus d'apprentissage lui-même (par exemple aux gradients calculés à chaque étape d'entraînement d'un réseau de neurones), de sorte que la présence ou l'absence d'un individu donné dans le jeu de données n'affecte que marginalement, de façon mathématiquement bornée, les prédictions du modèle final. Cette technique constitue une défense de fond contre l'inversion de modèle et l'attaque par appartenance, au prix d'un compromis à gérer avec soin : plus la garantie de confidentialité exigée est forte (plus le bruit ajouté est important), plus la performance prédictive du modèle tend à se dégrader — un arbitrage explicite entre confidentialité et utilité qui doit être calibré selon la sensibilité réelle des données concernées.

5. IA offensive : usage par les attaquants

- Génération automatisée de contenus de phishing personnalisés et sans fautes, à grande échelle.
- Assistance à l'écriture de code malveillant ou à l'automatisation de tâches de reconnaissance.
- Génération de deepfakes (voix, image, vidéo) utilisés dans des fraudes ciblées (ex. usurpation de la voix d'un dirigeant pour autoriser un virement frauduleux).
- Automatisation partielle de certaines phases de reconnaissance ou de rédaction de rapports par des attaquants.

5.1 Ce que l'IA générative change réellement dans la chaîne d'attaque

Il est utile de préciser en quoi l'IA générative modifie qualitativement (et pas seulement quantitativement) certaines étapes classiques d'une chaîne d'attaque (*kill chain*), plutôt que de se contenter d'un constat général :

- **Passage à l'échelle de la personnalisation** : historiquement, un phishing très personnalisé (*spear phishing*) exigeait un travail de reconnaissance manuel coûteux en temps, limitant ce niveau de sophistication à des cibles à forte valeur. L'automatisation partielle de cette reconnaissance et de la rédaction du message par des modèles génératifs abaisse ce coût, rendant potentiellement viable un phishing hautement personnalisé à une échelle beaucoup plus large.
- **Réduction de la barrière linguistique et technique** : un attaquant peu familier d'une langue cible, ou peu expérimenté techniquement, peut s'appuyer sur des outils génératifs pour produire un texte convaincant dans une langue qu'il ne maîtrise pas, ou pour obtenir de l'assistance sur des tâches de programmation qu'il ne saurait pas réaliser seul — abaissant ainsi, potentiellement, le niveau de compétence minimal requis pour certaines formes d'attaque.
- **Deepfakes audio/vidéo** : la baisse du coût et de la complexité technique nécessaires pour produire un contenu synthétique crédible (voix imitée, visage substitué) élargit le champ des attaques d'ingénierie sociale au-delà du texte écrit, vers des canaux (appel téléphonique, visioconférence) traditionnellement considérés comme plus fiables pour une vérification d'identité informelle.

Ce constat ne signifie pas que l'IA générative crée des catégories d'attaques entièrement nouvelles (le phishing, l'ingénierie sociale et l'usurpation d'identité préexistent largement à ces outils), mais qu'elle en

modifie l'échelle, le coût et l'accessibilité — une nuance importante pour calibrer une réponse défensive proportionnée plutôt que de céder à un discours excessivement alarmiste ou, à l'inverse, complaisant.

6. Contre-mesures et bonnes pratiques

Menace	Contre-mesure
Attaques adversariales	entraînement adversarial (inclure des exemples adversariaux dans l'entraînement), détection de perturbations anormales, limitation du taux de requêtes à une API de modèle
Empoisonnement de données	validation et traçabilité des sources de données d'entraînement, détection d'anomalies dans les données avant réentraînement
Extraction/inversion de modèle	limitation et surveillance des requêtes API, techniques de confidentialité différentielle
IA offensive (phishing, deepfakes)	sensibilisation renforcée, vérification hors bande des demandes sensibles (procédures « callback » avant tout virement), défenses multi-signaux ne reposant pas uniquement sur des indices textuels

6.1 L'entraînement adversarial : principe et limite fondamentale

L'**entraînement adversarial** (*adversarial training*), cité dans le tableau ci-dessus comme contre-mesure aux attaques par évocation, consiste à générer explicitement des exemples adversariaux pendant la phase d'entraînement et à les inclure dans le jeu de données d'apprentissage, de sorte que le modèle apprenne à correctement classer également ces versions perturbées. Cette technique améliore mesurablement la robustesse du modèle face aux types de perturbations utilisés pendant l'entraînement, mais comporte une limite conceptuelle importante à connaître : elle n'offre en général aucune garantie de robustesse face à des types de perturbations *non anticipés* lors de l'entraînement — un attaquant qui découvre une nouvelle méthode d'attaque non représentée dans le jeu d'entraînement adversarial peut potentiellement continuer à tromper le modèle. Elle s'apparente ainsi davantage à un durcissement empirique qu'à une preuve formelle de robustesse, ce qui justifie de la combiner systématiquement avec d'autres mesures (surveillance en production, limitation des requêtes, défense en profondeur) plutôt que de la considérer comme une solution définitive.

6.2 Vers une gouvernance de la sécurité de l'IA en entreprise

Au-delà des contre-mesures techniques ponctuelles, une organisation déployant des systèmes d'IA en production gagne à structurer une démarche de gouvernance dédiée, comparable dans son esprit à un programme de sécurité applicative classique mais adaptée aux spécificités du ML :

- **Inventaire des modèles en production** : savoir précisément quels modèles sont déployés, sur quelles données ils ont été entraînés, et quelles décisions ils influencent — un préalable simple mais souvent négligé.
- **Évaluation de robustesse avant mise en production**, incluant des tests adversariaux ciblés proportionnés à la criticité du système (un modèle de recommandation de contenu n'appelle pas

le même niveau d'exigence qu'un système de contrôle d'accès biométrique).

- **Plan de réponse à incident spécifique à l'IA**, anticipant les scénarios propres à ce domaine (détection d'une dérive de performance suspecte, procédure de retrait rapide d'un modèle compromis, communication en cas de fuite de données d'entraînement).
- **Formation continue des équipes**, tant côté développement (bonnes pratiques de robustesse et de confidentialité) que côté utilisateurs finaux (sensibilisation aux nouvelles formes d'ingénierie sociale assistées par IA).

Ce dernier point relie directement ce chapitre à l'ensemble du module : la sécurité de l'IA n'est pas un sujet isolé réservé aux data scientists, mais un prolongement naturel des pratiques de sécurité organisationnelle et technique déjà enseignées dans les autres modules du programme.

À retenir

- Les systèmes d'IA sont vulnérables à des classes d'attaques spécifiques (adversariales, empoisonnement, extraction) distinctes des vulnérabilités logicielles classiques, notamment parce que leur logique est apprise à partir de données plutôt qu'écrite explicitement.
- Les attaques adversariales exploitent des propriétés géométriques des modèles en haute dimension ; la transférabilité des exemples adversariaux entre modèles limite fortement l'intérêt défensif de la seule opacité (« security through obscurity »).
- L'empoisonnement de données peut viser une dégradation générale (facilement détectable) ou installer une porte dérobée ciblée (beaucoup plus difficile à détecter), avec un risque directement lié au mode d'entraînement (hors ligne vs apprentissage continu).
- Les attaques d'inversion de modèle exploitent le lien entre surapprentissage et fuite d'information ; la confidentialité différentielle offre une garantie mathématique, au prix d'un compromis explicite avec la performance.
- L'IA offensive ne crée pas nécessairement de nouvelles catégories d'attaques, mais en réduit le coût et augmente l'échelle et le réalisme (phishing personnalisé, deepfakes), ce qui impose des contre-mesures organisationnelles autant que techniques.
- L'entraînement adversarial améliore la robustesse empirique mais ne garantit pas une protection contre des attaques non anticipées ; une véritable gouvernance de la sécurité de l'IA en entreprise dépasse les seules contre-mesures techniques ponctuelles.